03

MGU BCA - 3RD SEM

Python Programming

Module 3 - Files, Exceptions, Visualization

By Aben Sojan

Exam - 16 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

Exam: MG3SECBCA200 Python Programming - 50 marks, 1 hour 30 minutes (MGU Honours model paper, mgu.ac.in, Oct 2025).

Part A: 10 x 2 = 20 marks, answer ALL - short questions from every module.

Part B: answer any 3 of 5 x 5 = 15 marks - programs and detailed concept answers.

Part C: answer any 1 of 2 x 15 = 15 marks - one long essay or program.

Module 3 in the model paper: text vs binary file, try block (Part A); stacked vs grouped bar chart, 2D vs 3D plotting (Part B).

TOPIC 1

Files: Types and Basic Operations

1

Text files store readable characters (.txt); binary files store raw bytes (images, .dat).

open("a.txt", "r") reads; "w" writes and erases old content; "a" adds at the end; "b" means binary mode.

Always close files with close(), or use with open(...) which closes automatically.

read() reads everything; readline() reads one line; readlines() gives a list of lines.

write() writes a string; writelines() writes a list of strings.

QUESTION BANK

1 Explain the write(), writeline() and writelines() methods with examples. (Nov 2025)

REPEATED

write(s) writes a single string to the file; it does not add a newline by itself.

writelines(lines) writes a list of strings one after another, also without adding newlines.

Note: the file object has no writeline() method; the singular form in the question means write().

Example:

```
f = open("data.txt", "w")
f.write("Hello\n")
f.writelines(["Line1\n", "Line2\n"])
f.close()
```

Always close the file (or use with) so the data is actually saved to disk.

write() is for one string, writelines() for a list, and you add newlines yourself where needed.

2 Explain the different file opening modes in Python.

IMPORTANT

A file mode is the second argument to open() and tells Python what you want to do with the file.

"r" opens for reading only; the file must already exist, otherwise an error occurs.

"w" opens for writing; it creates the file if missing and erases old content if it exists.

"a" opens for appending; new data is added at the end and old content stays safe.

"r+", "w+" and "a+" allow both reading and writing; w+ erases the file while a+ appends to it.

Add "b" for binary mode, used for images and pickle files: "rb" reads bytes, "wb" writes bytes.

"x" creates a new file and fails if the file already exists, which avoids accidental overwrites.

Example code:

```
f = open("data.txt", "w")
f.write("Hello")
f.close()
```

Choosing the right mode protects your data, so always pick the mode that matches your task.

3 Write a program to read a file and count lines, words and characters.

IMPORTANT

The task is to open a text file, read its content, and count its lines, words and characters.

The with statement opens the file and closes it automatically when the block ends.

read() brings the whole file content into one string called text.

splitlines() cuts the string at every newline, so the length of that list is the line count.

split() cuts the string at every whitespace, so the length of that list is the word count.

len(text) counts every character, including spaces and newline characters.

Example code:

```
with open("data.txt", "r") as f:
text = f.read()
lines = text.splitlines()
words = text.split()
print("Lines:", len(lines))
print("Words:", len(words))
print("Characters:", len(text))
```

Each counter uses the natural Python way of measuring that unit of text.

4 Why is the with statement preferred for files?

IMPORTANT

When you open a file, Python must later close it to free the file and save buffered data.

If you forget close(), buffered writes may never reach the file, so data is lost silently.

If an error happens between open and close, the file stays open and locked.

with open("data.txt", "r") as f: opens the file and assigns it to f.

When the block ends, Python closes the file automatically, even if an error occurred inside.

No explicit close() call is needed, so the code is shorter and cleaner.

Example code:

```
with open("marks.txt") as f:
data = f.read()
```

Once the block finishes, the file is guaranteed to be closed.

Because with is both safer and shorter, it is the preferred way to work with files.

5 Define a text file and a binary file.

IMPORTANT

A text file stores human-readable characters (letters, digits, symbols) encoded as text, for example .txt, .py and .csv files.

A binary file stores raw bytes that are not meant to be read as text, for example images, audio and .pyc files.

Text files are opened in 'r' or 'w' modes; binary files need 'rb' or 'wb' modes.

Reading a binary file as text garbles the data, so the open mode must match the content.

Text files hold characters, binary files hold raw bytes, and the mode must match.

6 Write a Python script to open a file, write a few lines of text into it and then read its content.

IMPORTANT

Open in 'w' to write, close it, then reopen in 'r' to read back.

```
with open("notes.txt", "w") as f:
f.write("Line one\n")
f.write("Line two\n")
with open("notes.txt", "r") as f:
print(f.read())
```

The with statement closes the file automatically, so nothing is lost.

read() returns the whole content as one string; the print shows both lines.

Write mode creates the content, read mode brings it back.

7 **Write a program to copy a text file to another file.**

IMPORTANT

Read the source fully, then write that content into the destination file.

```
with open("source.txt", "r") as src:
data = src.read()
with open("copy.txt", "w") as dst:
dst.write(data)
print("Copied")
```

For big files copy line by line instead: for line in src: dst.write(line).

Read-all then write-all copies any text file exactly.

8 **Explain how you read and write numbers from/to a file with an example.**

IMPORTANT

Files store text, so numbers must be converted with str() when writing and int() or float() when reading.

```
with open("nums.txt", "w") as f:
f.write(str(42) + "\n")
with open("nums.txt", "r") as f:
n = int(f.read().strip())
print(n + 8)   # 50, proves n is a real number
```

Writing the raw number without str() raises a TypeError.

strip() removes the newline before int() converts the text back.

str() on the way out and int() or float() on the way in.

TOPIC 2

Pickle, CSV and JSON

2

pickle saves Python objects to a binary file: pickle.dump(obj, f) writes, pickle.load(f) reads back.

CSV files store table data separated by commas; the csv module gives reader and writer.

JSON stores data as readable text like {"name": "Aben"}; json.dump() writes, json.load() reads.

CSV suits spreadsheets and Excel; JSON suits web data and settings files.

QUESTION BANK

9 What is pickling? Explain dump() and load() with an example.

IMPORTANT

Pickling converts a Python object into a byte stream that can be stored in a file.

This lets you save complex objects like lists and dictionaries and use them again later.

Unpickling is the reverse: it reads the bytes back and rebuilds the original object.

Both steps need the pickle module and the file opened in binary mode ("wb" or "rb").

pickle.dump(obj, f) pickles obj and writes it to file f.

pickle.load(f) reads the next pickled object from file f and returns it.

Example code:

```
import pickle
data = {"name": "Aben", "marks": 95}
with open("data.pkl", "wb") as f:
pickle.dump(data, f)
with open("data.pkl", "rb") as f:
saved = pickle.load(f)
print(saved)
```

The dictionary comes back unchanged with all its keys and values.

Pickling is ideal for saving Python objects quickly, but pickle files are only safe with trusted data.

10 How do you read and write CSV files in Python?

IMPORTANT

CSV (Comma Separated Values) files store table data with commas between values and lines between rows.

The csv module handles reading and writing, including tricky cases like commas inside values.

To write: open the file with newline="" to avoid blank lines, then get a writer with csv.writer(f).

writerow([..]) writes one row from a list; writerows([..]) writes many rows at once.

To read: use csv.reader(f), which gives each row back as a list of strings.

Example code:

```
import csv
with open("marks.csv", "w", newline="") as f:
```

```
w = csv.writer(f)
w.writerow(["name", "marks"])
w.writerow(["Aben", 95])
with open("marks.csv", "r") as f:
for row in csv.reader(f):
print(row)
```

Every row arrives as a list like ["Aben", "95"], so convert numbers with int() when needed.

CSV is the standard way to exchange simple tables with Excel and other tools.

11 How do you read and write JSON files? Distinguish JSON from CSV.

IMPORTANT

JSON (JavaScript Object Notation) stores data as readable text in name-value form, like Python dictionaries.

json.dump(obj, f) writes a Python object such as a dict or list into a text file.

json.load(f) reads the file and rebuilds the Python object.

Example code:

```
import json
d = {"name": "Aben", "marks": 95}
with open("d.json", "w") as f:
json.dump(d, f)
with open("d.json", "r") as f:
print(json.load(f))
```

The file contains plain readable text like {"name": "Aben", "marks": 95}.

Difference 1: CSV holds flat row-column tables only, while JSON holds nested structures like lists inside dicts.

Difference 2: CSV is best for spreadsheet-style data and Excel; JSON is best for web data, APIs and settings files.

Difference 3: CSV rows are all plain strings, while JSON preserves types like numbers, booleans and lists.

TOPIC 3

3

Exception Handling

An exception is a runtime error like dividing by zero or opening a missing file.

try: holds risky code; except: runs if an error happens; else: runs if no error; finally: always runs.

raise creates your own error: raise ValueError("bad age").

Hierarchy: BaseException at the top, then Exception, then ValueError, TypeError, ZeroDivisionError, FileNotFoundError and others.

Catch specific errors first and the general Exception last.

QUESTION BANK

12 **Explain try-except-else-finally with an example.**

IMPORTANT

An exception is a runtime error, such as dividing by zero or converting bad input, that crashes the program if unhandled.

try: holds the risky code that might fail.

except: runs only when an error happens; you can name the error type, like ZeroDivisionError, to catch.

else: runs only when no error happened in the try block, for code that depends on success.

finally: runs in every case, error or not, and is used for cleanup like closing files.

Example code:

```
try:
x = int(input("Enter a number: "))
print(10 / x)
except ZeroDivisionError:
print("Cannot divide by zero")
except ValueError:
print("Enter digits only")
else:
print("Division done")
finally:
print("Program over")
```

Zero input prints the friendly zero message instead of crashing.

Using all four blocks gives full control: handle errors, confirm success, and always clean up.

13 What is the raise statement? Give an example.

IMPORTANT

Sometimes your program must fail on purpose, like when input breaks your own rules.

The raise statement creates an exception at the exact point you choose.

Syntax: raise ErrorType("message"), for example raise ValueError("age cannot be negative").

After raise, Python stops the normal flow and jumps to the nearest matching except block.

Example code:

```
age = int(input("Enter age: "))
if age < 0:
raise ValueError("age cannot be negative")
print("Age:", age)
```

A negative age raises ValueError with the given message instead of continuing silently.

raise is used for validation: reject bad data early with a clear error message.

14 Explain the exception hierarchy in Python.

IMPORTANT

Exceptions in Python are organized as a class tree, where every error is an object of some class.

BaseException is the root of the tree; almost everything inherits from it.

Exception is the main branch, and all normal runtime errors hang below it.

Built-in errors include: ValueError (wrong value), TypeError (wrong type), ZeroDivisionError (divide by zero), FileNotFoundError (missing file), IndexError (bad list index), KeyError (missing dict key).

Your own errors are made by writing a class that extends Exception, like class MyError(Exception): pass.

An except block catches the named class and all its subclasses, so except Exception catches almost everything.

Because of this, always catch specific errors first and put the general Exception last, or it would swallow the specific ones.

Example: except ValueError catches only bad values, while a bare except Exception at the end catches the rest.

15 **Write a program that handles division by zero gracefully.**

IMPORTANT

Goal: read two numbers, divide them, and show a friendly message if the divisor is zero.

Wrap the division inside a try block so errors are caught instead of crashing the program.

except ZeroDivisionError catches division by zero and prints a friendly message.

except ValueError catches non-numeric input from int().

Example code:

```
try:
a = int(input("Enter numerator: "))
b = int(input("Enter denominator: "))
print("Result:", a / b)
except ZeroDivisionError:
print("Error: cannot divide by zero")
except ValueError:
print("Error: enter numbers only")
```

Entering 0 as the denominator prints "Error: cannot divide by zero" and the program ends calmly.

Handling errors this way makes programs user friendly instead of showing scary tracebacks.

TOPIC 4

Data Visualization

4

matplotlib.pyplot, imported as plt, draws charts in Python.

plt.plot(x, y) draws a line graph; plt.show() displays the figure.

plt.bar() draws bar charts; plt.hist() draws histograms; plt.pie() draws pie charts.

Sine and cosine curves: y = np.sin(x) with x = np.linspace(0, 2 * np.pi, 100).

3D plots need from mpl_toolkits.mplot3d import Axes3D, then ax.plot_surface().

QUESTION BANK

16 How do you draw a line plot, bar chart and pie chart in matplotlib?

IMPORTANT

matplotlib.pyplot, usually imported as plt, is Python's standard library for drawing charts.

plt.plot(x_values, y_values) draws a line plot joining the given points.

plt.bar(categories, values) draws a bar chart with one bar per category.

plt.pie(values, labels=labels) draws a pie chart showing each part of a whole.

plt.show() displays the figure on screen; call it after setting up each chart.

Example code:

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [10, 20, 15])
plt.show()
plt.bar(["A", "B", "C"], [5, 8, 3])
plt.show()
plt.pie([40, 30, 30], labels=["A", "B", "C"])
plt.show()
```

plt.xlabel(".."), plt.ylabel(".") and plt.title("..") label any chart and make it exam worthy.

Line plots show trends over time, bar charts compare quantities, and pie charts show shares of a whole.

17 Write code to plot sine and cosine curves on the same graph.

IMPORTANT

To draw smooth curves we need many x points, which np.linspace(0, 2 * np.pi, 100) gives.

np.sin(x) and np.cos(x) compute the sine and cosine of every point at once.

Calling plt.plot twice draws both curves on the same figure.

The label argument names each curve so the legend can identify them.

plt.legend() displays the label box on the graph.

Example code:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0, 2 * np.pi, 100)
plt.plot(x, np.sin(x), label="sin")
plt.plot(x, np.cos(x), label="cos")
plt.legend()
plt.show()
```

The result is a smooth sine wave and a smooth cosine wave drawn together.

The same pattern works for any two curves: compute points, plot each with a label, add the legend.

18 Write a short note on histograms.

IMPORTANT

A histogram is a bar-like chart that shows how numerical values are distributed across ranges.

plt.hist(data, bins=10) splits the data range into 10 equal bins and counts the values in each.

Each bar's height shows how many values fell into that range.

Tall bars reveal where most values cluster, and the shape reveals patterns like normal or skewed distributions.

Example code:

```
import matplotlib.pyplot as plt
marks = [45, 67, 78, 56, 90, 34, 72, 88, 61, 55]
plt.hist(marks, bins=5)
plt.xlabel("Marks")
plt.ylabel("Students")
plt.title("Distribution of Marks")
plt.show()
```

Where a bar chart compares fixed categories, a histogram discovers the shape of continuous data.

Histograms are widely used in statistics and data analysis to summarize large datasets.

19 Differentiate between a stacked bar chart and a grouped bar chart.

IMPORTANT

A grouped bar chart places bars for each category side by side, so values are compared directly.

A stacked bar chart piles the values of one category on top of another in a single bar, showing the total and its parts.

Example:

```
import matplotlib.pyplot as plt
x = ["A", "B"]
plt.bar(x, [5, 7])   # grouped-style: separate bars per value
plt.show()
```

Stacked bars suit part-to-whole views like sales split by region; grouped bars suit direct comparison.

Grouped means side by side for comparison, stacked means piled for totals and shares.

20 **Compare 2D and 3D plotting in Python with examples.**

IMPORTANT

2D plotting draws on flat x-y axes using matplotlib.pyplot functions like plot(), bar() and pie().

3D plotting adds a z axis through mpl_toolkits.mplot3d for surfaces and 3D scatter views.

Example:

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [4, 5, 6])   # 2D line plot
plt.show()
```

2D covers most exam charts; 3D is used when the data truly has three dimensions to show.

2D is the everyday x-y chart, 3D adds depth for three-variable data.

21 **Define the turtle module in Python. What are the steps in plotting a diagram using turtle? Give an example.**

IMPORTANT

The turtle module gives a pen (the turtle) that draws lines as it moves, used for simple 2D graphics.

Steps: import turtle, create a pen, move it with forward(), left() and right(), then keep the window open.

Example:

```
import turtle
t = turtle.Turtle()
for _ in range(4):
t.forward(100)
t.left(90)      # draws a square
turtle.done()
```

penup() and pendown() control whether movement draws; color() and pensize() style the line.

Turtle turns movement commands into drawings, step by step.

TOPIC 5

5

Practical Programs

All 14 syllabus practical programs are given below as complete working code.

Type each program, run it, and check the output before the exam.

In the exam, write the code neatly with correct indentation.

QUESTION BANK

22 **Write a Python program to check whether a number is prime.**

IMPORTANT

Logic: read a number and test whether any value from 2 up to its square root divides it.

If a divisor is found, the number is not prime and the loop stops early with break.

If the loop finishes without a divisor (the for-else), the number is prime.

A prime number is divisible only by 1 and itself.

Full program:

```
n = int(input("Enter a number: "))
if n < 2:
print(n, "is not prime")
else:
for i in range(2, int(n ** 0.5) + 1):
if n % i == 0:
print(n, "is not prime")
break
else:
print(n, "is prime")
```

Checking only up to the square root keeps it fast.

23 Write a Python program to find the power of a number using a function.

IMPORTANT

Logic: define a function power(base, exp) that multiplies base by itself exp times.

Start result at 1 and repeat the multiplication in a loop exp times.

The main code reads base and exponent, calls the function and prints the result.

Full program:

```
def power(base, exp):
result = 1
for i in range(exp):
result = result * base
return result
b = int(input("Enter base: "))
e = int(input("Enter exponent: "))
print("Result:", power(b, e))
```

Example: base 2, exponent 5 gives 32.

24 Write a Python function to check whether a number is even or odd.

IMPORTANT

Logic: a number is even when its remainder on division by 2 is zero.

The function check uses the % operator and returns the word even or odd.

The main code reads a number, calls check and prints the verdict.

Full program:

```
def check(num):
if num % 2 == 0:
return "even"
else:
return "odd"
n = int(input("Enter a number: "))
print(n, "is", check(n))
```

The % operator gives the remainder; remainder 0 means even.

25 Write a Python program to find the sum of the alternating series 1 - 2 + 3 - 4 ... n.

IMPORTANT

Logic: the series 1 - 2 + 3 - 4 ... adds odd terms and subtracts even terms.

Loop i from 1 to n, adding i when i is odd and subtracting i when i is even.

Example: for n = 5 the sum is 1 - 2 + 3 - 4 + 5 = 3.

The series 1 - 2 + 3 - 4 ... n adds odd terms and subtracts even terms.

Full program:

```
n = int(input("Enter n: "))
total = 0
for i in range(1, n + 1):
if i % 2 == 0:
total = total - i
else:
total = total + i
print("Sum of series:", total)
```

For n = 5 the answer is 1 - 2 + 3 - 4 + 5 = 3.

26 Write a menu-driven Python program for string operations: length, uppercase, reverse and vowel count.

IMPORTANT

Logic: read a string once, then ask the user to pick one of four operations from a menu.

Use if-elif to run length, uppercase, reverse (slicing s[::-1]) or vowel counting.

The vowel counter walks through the string and counts characters found in aeiouAEIOU.

Full program:

```
s = input("Enter a string: ")
print("1. Length  2. Uppercase  3. Reverse  4. Count vowels")
ch = int(input("Enter choice: "))
if ch == 1:
print("Length:", len(s))
elif ch == 2:
print(s.upper())
elif ch == 3:
print(s[::-1])
elif ch == 4:
```

```
vowels = "aeiouAEIOU"
count = 0
for c in s:
if c in vowels:
count = count + 1
print("Vowels:", count)
else:
print("Invalid choice")
```

s[::-1] is the shortcut for reversing a string.

27 **Write a Python program to find all occurrence indices of a character in a string.**

IMPORTANT

Logic: scan the string position by position and compare each character with the target.

Every matching index is printed immediately; a flag records whether anything matched.

If the flag is still False at the end, report that the character was not found.

Full program:

```
s = input("Enter a string: ")
ch = input("Enter character to find: ")
found = False
for i in range(len(s)):
if s[i] == ch:
print("Found at index:", i)
found = True
if not found:
print("Character not found")
```

The loop checks every index and prints each match.

28 **Write a menu-driven Python program using Numpy to find sum, mean, max and min of an array.**

IMPORTANT

Logic: store the data in a Numpy array once, then offer sum, mean, max and min on a menu.

The choice number selects which Numpy function (np.sum, np.mean, np.max, np.min) to run.

Numpy computes each statistic over the whole array in a single call.

Full program:

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print("1. Sum  2. Mean  3. Max  4. Min")
ch = int(input("Enter choice: "))
if ch == 1:
print("Sum:", np.sum(arr))
elif ch == 2:
print("Mean:", np.mean(arr))
elif ch == 3:
print("Max:", np.max(arr))
elif ch == 4:
print("Min:", np.min(arr))
else:
print("Invalid choice")
```

Numpy functions work on the whole array at once.

29 **Write a Python program to count the frequency of each letter in a string using a dictionary.**

IMPORTANT

Logic: build a dictionary mapping each letter to its count.

Walk through the lowercased string, skipping spaces; add a new key for unseen letters or add 1 for seen ones.

Finally print every letter with its count from the dictionary.

Full program:

```
s = input("Enter a string: ").lower()
freq = {}
for c in s:
```

```
if c != " ":
if c in freq:
freq[c] = freq[c] + 1
else:
freq[c] = 1
for letter, count in freq.items():
print(letter, ":", count)
```

lower() makes counting case insensitive; spaces are skipped.

30 **Write a Python program to demonstrate tuple operations: length, max, min, sum, index, count and sorting.**

IMPORTANT

Logic: create a fixed tuple and demonstrate the operations allowed on it.

Built-ins len, max, min and sum work directly; index and count search the tuple.

sorted() returns a list, so wrap it in tuple() to get a sorted tuple back.

Full program:

```
t = (5, 2, 9, 1, 7)
print("Tuple:", t)
print("Length:", len(t))
print("Max:", max(t), "Min:", min(t))
print("Sum:", sum(t))
print("Index of 9:", t.index(9))
print("Count of 2:", t.count(2))
print("Sorted:", tuple(sorted(t)))
```

sorted() returns a list, so tuple() converts it back.

31 **Write a Python program to copy alternate lines from one file to another.**

IMPORTANT

Logic: read all lines of the source file into a list with readlines().

Indices 0, 2, 4, ... are the alternate lines, picked with range(0, len(lines), 2).

Write only those lines into the destination file.

Full program:

```
with open("source.txt", "r") as f1:
lines = f1.readlines()
```

```
with open("dest.txt", "w") as f2:
for i in range(0, len(lines), 2):
f2.write(lines[i])
print("Alternate lines copied to dest.txt")
```

range(0, len(lines), 2) picks indices 0, 2, 4, the alternate lines.

32 **Write a Python program to handle ZeroDivisionError using exception handling.**

IMPORTANT

Logic: read numerator and denominator and divide them inside a try block.

If the denominator is zero, ZeroDivisionError is caught and a friendly message prints.

ValueError handles the case where the user types non-numeric input.

Full program:

```
try:
a = int(input("Enter numerator: "))
b = int(input("Enter denominator: "))
print("Result:", a / b)
except ZeroDivisionError:
print("Error: cannot divide by zero")
except ValueError:
print("Error: enter numbers only")
```

The specific ZeroDivisionError is caught before the general ValueError case.

33 **Write a Python program that defines a custom exception for duplicate entries and handles it.**

IMPORTANT

Logic: define DuplicateError as a class that extends Exception.

While collecting items, check each new entry against the list collected so far.

If an entry repeats, raise DuplicateError, which the except block catches and reports.

A custom exception is made by writing a class that extends Exception.

Here DuplicateError is raised when the same item is entered twice.

Full program:

```
class DuplicateError(Exception):
pass
items = []
```

```
n = int(input("How many items: "))
try:
for i in range(n):
x = input("Enter item: ")
if x in items:
raise DuplicateError("Duplicate entry: " + x)
items.append(x)
print("List:", items)
except DuplicateError as e:
print("Error:", e)
```

raise triggers the exception; except catches it by name.

34 Write a Python program to plot sine and cosine curves using matplotlib.

IMPORTANT

Logic: create 100 smooth x points from 0 to 2 pi with np.linspace.

Plot np.sin(x) and np.cos(x) on the same figure with labels, then add legend, grid and titles.

plt.show() displays both curves together.

Full program:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0, 2 * np.pi, 100)
plt.plot(x, np.sin(x), label="sin(x)")
plt.plot(x, np.cos(x), label="cos(x)")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Sine and Cosine Curves")
plt.legend()
plt.grid(True)
plt.show()
```

linspace gives 100 smooth points between 0 and 2 pi.

35 **Write a Python program to draw a line plot of monthly profit.**

IMPORTANT

Logic: store month names and matching profit values in two parallel lists.

plt.plot joins the monthly points with a line; marker="o" marks each data point with a dot.

Labels and a title explain the axes before plt.show() displays the chart.

Full program:

```
import matplotlib.pyplot as plt
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
profit = [12000, 15000, 13000, 18000, 20000, 17000]
plt.plot(months, profit, marker="o")
plt.xlabel("Month")
plt.ylabel("Profit (Rs)")
plt.title("Monthly Profit")
plt.grid(True)
plt.show()
```

marker="o" puts a dot on each data point.